

Avaliação Prática

Contrato ViaSerra ·

O cenário

A ViaSerra Transportes opera linhas intermunicipais de ônibus entre a serra gaúcha e o litoral, com 18 mil passageiros por dia. É pelo portal da empresa que o passageiro consulta horários e recarrega o cartão de passagem.

O fornecedor que cuidava do portal encerrou o contrato e deixou só os arquivos do site em HTML e uma página de manutenção que nunca funcionou. Ninguém na ViaSerra sabe como colocar isso no ar de novo.

A TechInova Serviços assumiu o contrato e escalou você. A diretoria quer ver o portal funcionando daqui a duas horas.

Sua missão

Empacotar o portal numa imagem Docker sua, publicar no Docker Hub, consertar a página de manutenção e, no fim, subir os dois sites de uma vez com um `docker-compose.yml` que já vem quase pronto.

Você não vai escrever nenhuma linha de programação. Tudo se resolve com Dockerfile, HTML (só o rodapé), um `docker-compose.yml` com lacunas e Git.

Regras

1. O trabalho é individual. Dockerfiles idênticos entre alunos serão tratados como cópia, para os dois lados.
2. O repositório no GitHub e o repositório no Docker Hub precisam estar públicos na hora da correção.
3. O arquivo `.env` nunca pode aparecer no histórico do Git. Se aparecer, a parte de Git vale zero.
4. Faça commit ao fim de cada parte. Um único commit no fim das 2 horas perde os pontos de histórico.

Seus valores personalizados

As portas e a tag da imagem dependem dos dois últimos dígitos da sua matrícula (XX). O verificador confere esses valores, e o professor também.

O quê	Regra	Exemplo (matrícula 20250137)
Porta do portal	8000 + XX	8037
Porta da página de manutenção	7000 + XX	7037
Imagem do portal	<code><usuario-dockerhub>/viaserra-portal:1.0-<matrícula></code>	<code>ana.souza/viaserra-portal:1.0-20250137</code>
Repositório GitHub	<code>avaliacao-docker-<matrícula></code> , público	<code>avaliacao-docker-20250137</code>

Tempo sugerido e pontuação

Parte	Pontos
0 · Preparação do repositório	dentro de Git
1 · Dockerfile do portal	2,5
2 · Publicação no Docker Hub	1,5
3 · Conserto da página de manutenção	2,5
4 · Primeiro docker-compose	1,0
5 · Verificador, respostas e entrega	Git 1,0 · respostas 1,5
Total	10,0

Parte 0 · Preparação do repositório

Crie no GitHub um repositório público vazio chamado **avaliacao-docker-<matrícula>**, sem README e sem .gitignore. Descompacte o pacote e trabalhe dentro da pasta dele.

```
cd avaliacao-docker-viaserra
git init
git branch -M main
git remote add origin https://github.com/<seu-usuario>/avaliacao-docker-<matricula>.git

# faça login no Docker Hub já: o laboratório inteiro sai pelo mesmo IP
docker login -u <seu-usuario-dockerhub>
```

Antes do primeiro commit:

1. Crie um arquivo **.gitignore** na raiz com uma linha: **.env**.
2. Copie **.env.example** para **.env** e preencha **MATRICULA** e **DOCKERHUB_USER**. O verificador lê esses dois valores.
3. Rode **git status** e confira que o **.env** não aparece. Só então faça **git add .** e o commit **chore: estrutura inicial**.
4. Faça o push para a **main**.

Por que isso vem primeiro

Se o **.env** entrar num commit, apagar o arquivo depois não resolve: a senha continua no histórico do Git, visível para qualquer pessoa.

Parte 1 · Dockerfile do portal

O site está pronto em `portal/html/` (um `index.html` e um `estilo.css`). Abra o `index.html` num editor e troque, no rodapé, **SEU NOME AQUI** pelo seu nome e `00000000` pela sua matrícula. Não mexa em mais nada do HTML.

Depois crie o arquivo `portal/Dockerfile` atendendo aos requisitos:

#	Requisito
R1.1	Imagem base oficial do Nginx com tag fixa. <code>latest</code> ou imagem sem tag não são aceitas.
R1.2	O conteúdo de <code>portal/html/</code> copiado para a pasta de onde o Nginx serve os arquivos.
R1.3	Porta 80 documentada com <code>EXPOSE</code> .
R1.4	Uma instrução <code>LABEL</code> com o seu nome e a sua matrícula.

Construa com o nome definitivo da imagem e teste antes de publicar:

```
docker build -t <usuario>/viaserra-portal:1.0-<matricula> ./portal
docker run -d --name teste-portal -p <8000+XX>:80 <usuario>/viaserra-portal:1.0-<matricula>
# abra http://localhost:<8000+XX> e confira seu nome no rodapé
docker rm -f teste-portal
```

Apareceu "Welcome to nginx!"?

O container está funcionando, mas servindo a página padrão da imagem. Seu arquivo foi parar na pasta errada. Use `docker exec` para olhar dentro do container.

Commit: `feat: Dockerfile do portal`. Responda as perguntas 1 e 2 do `respostas.md`.

Parte 2 · Publicação no Docker Hub

1. No Docker Hub, crie o repositório público `viaserra-portal`.
2. Faça o push da imagem com a tag que você já construiu.
3. Abra a página do repositório numa aba anônima do navegador e confira que a tag aparece.

```
docker push <usuario>/viaserra-portal:1.0-<matricula>
```

Na correção, o professor baixa a sua imagem em outra máquina. Imagem privada ou com tag fora da regra vale zero nesta parte. Responda as perguntas 3 e 4.

Parte 3 · A página de manutenção

Quando o portal sair do ar para manutenção, a ideia é trocar o container por uma página simples de "Voltamos em breve". O fornecedor deixou a página pronta em `manutencao/site/index.html` e este Dockerfile:

```
# Página de manutenção do portal, deixada pelo fornecedor anterior.  
# Deveria mostrar o aviso "Voltamos em breve" na porta 80 do container.
```

```
FROM nginx:1.27-alpine
```

```
WORKDIR /usr/share/nginx
```

Há três defeitos. Eles aparecem um de cada vez: o segundo só se mostra depois que você corrige o primeiro. Construa e rode com os comandos abaixo, observe o que acontece, corrija e repita.

```
docker build -t manutencao:<matricula> ./manutencao  
docker run -d --name manut -p <7000+XX>:80 manutencao:<matricula>  
docker ps -a          # o container está de pé?  
docker logs manut     # o que ele disse?  
docker rm -f manut    # antes de testar de novo
```

Critério de aceite

Em `http://localhost:<7000+XX>` aparece a página laranja com "Voltamos em breve", e o container continua em execução em `docker ps`.

Quando funcionar, remova o container de teste com `docker rm -f manut`, porque a Parte 4 usa a mesma porta. Preencha a tabela da pergunta 5 com os defeitos que encontrou. Cada defeito real vale ponto; cada "defeito" listado que não é defeito desconta 0,1. Responda também a pergunta 6.

Commit: `fix: corrige Dockerfile da página de manutenção`.

Parte 4 · Primeiro docker-compose

Até aqui você subiu cada container com um `docker run` comprido. O Docker Compose faz a mesma coisa a partir de um arquivo: você descreve os containers uma vez e sobe todos com um único comando. O assunto vai ser estudado com calma nas próximas aulas. Hoje é só um primeiro contato, e o arquivo já vem pronto no pacote, faltando quatro valores:

```
services:  
  
  portal:  
    image: _____ # a imagem que você publicou na Parte 2 (usuario/nome:tag)  
    ports:  
      - "_____:80"    # porta do host: 8000 + XX  
    restart: unless-stopped  
  
  manutencao:  
    build: _____ # a pasta onde está o Dockerfile que você corrigiu na Parte 3  
    ports:  
      - "_____:80"    # porta do host: 7000 + XX  
    restart: unless-stopped
```

Compare com o que você já conhece: `image` é o nome que vem no fim do `docker run`, `ports` é o mesmo `-p host:container`, e `build` aponta para a pasta que você passava no `docker build`. Troque cada `___` pelo valor certo, salve e suba:

```
docker compose up -d      # constrói a manutenção e sobe os dois containers
docker compose ps         # os dois devem aparecer como running
docker compose logs       # se algo der errado, a resposta costuma estar aqui
```

Cuidado com os espaços

No YAML, a quantidade de espaços no começo da linha faz parte da sintaxe. Não use Tab e não mude a indentação do arquivo. Se aparecer `invalid hostPort: ___`, ainda tem lacuna sem preencher.

Abra o portal em `http://localhost:<8000+XX>` e clique no botão "Aviso de manutenção": ele leva para a página da Parte 3. Commit: `feat: docker-compose com portal e manutenção`. Responda as perguntas 7 e 8.

Parte 5 · Verificador e entrega

Com o `docker compose` no ar, rode o verificador da raiz do projeto:

```
# Linux ou Git Bash
bash scripts/verificar.sh

# ou no Windows PowerShell
powershell -ExecutionPolicy Bypass -File scripts\verificar.ps1
```

O verificador faz 9 checagens e diz o que falhou, nunca como consertar. Quando tudo passar, ele imprime um código de conclusão único para a sua matrícula. Cole o código na pergunta 9 e tire um print da tela.

1. Termine o `respostas.md` e faça o commit `docs: respostas da avaliação`.
2. Crie a tag e envie: `git tag v1.0` e depois `git push origin main --tags`.
3. Confira no GitHub, numa aba anônima, que o repositório está público e que o `.env` não aparece.
4. Envie pelo Google Classroom os dois links (repositório no GitHub e repositório no Docker Hub) e o print do resultado do verificador.

Critérios de avaliação

Parte	O que é avaliado	Pontos
Git e GitHub (0 e 5)	<code>.gitignore</code> correto, <code>.env</code> fora do histórico, um commit por parte com mensagem que diz o que mudou, tag <code>v1.0</code>	1,0
1 · Dockerfile do portal	R1.1 a R1.4, rodapé editado e imagem funcionando	2,5
2 · Docker Hub	Imagem pública, tag na regra, roda na máquina do professor	1,5
3 · Página de manutenção	Imagem corrigida funcionando e tabela de defeitos com o que você viu acontecer (falso positivo desconta 0,1)	2,5

Parte	O que é avaliado	Pontos
4 · Primeiro docker-compose	As quatro lacunas preenchidas e os dois sites no ar com <code>docker compose up -d</code>	1,0
Respostas	Perguntas coerentes com o que está no seu repositório	1,5